SIMATS ENGINEERING

Department of _____

ASSIGNMENT 3 – CO3

Constructors, Destructors and Operator Overloading

Total Marks: 50

Course: Object Oriented Programming with C++
Course Code: DSA01

Student Name	: Krishnan Gupta M
Register Number	: 192524062
Faculty Name	: Arun Kumar
Submission Date	: 08 November 2026
GitHub Repository	: https://github.com/192524062simats-ops/DSA0103c-/tree/main/c%2B%2B%20assignment

TABLE OF CONTENTS

Q1. Constructors and Constructor Overloading (6 Marks).....3
Q2. Copy Constructors: Shallow and Deep Copies (7 Marks).....8
Q3. Dynamic Constructors and Destructors (7 Marks).....13
Q4. Arithmetic and Stream Operator Overloading (7 Marks).....19
Q5. Increment, Comparison and Subscript Operators (6 Marks).....23
Q6. Rules of Operator Overloading (7 Marks).....27
Q7. Complex Number Class (10 Marks).....31
Overall Learning Outcomes.....36
Final Conclusion.....36
GitHub Repository Link.....36
References.....36

Q1. Constructors and Constructor Overloading (6 Marks)

1.1 Aim

To write a C++ program using a Book class that demonstrates default, parameterised and overloaded constructors, constructor delegation and member initializer lists, and to make sure an object is never created with a negative number of copies.

1.2 Problem Description

A Book class stores the title, author and number of copies of a book. Objects must be creatable with no arguments, with a title only, with a title and author, and with all three values. Missing values take defaults ("Unknown" for text and 0 for copies), and a negative number of copies must be rejected.

1.3 Concepts Used

- Default constructor
- Parameterised constructor
- Constructor overloading
- Constructor delegation
- Member initializer list
- explicit keyword
- Exception handling (try / catch) for invalid input
- Member initialisation order and the -Wreorder warning

1.4 Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode for your program here.

1.5 C++ Source Code

The source code should include the following parts:

- Default constructor
- Parameterised constructor
- Overloaded constructors
- Delegating constructor
- Member initializer lists
- explicit constructor demonstration
- main() with the invalid-copies test

```
// Paste your C++ source code here
```

1.6 Program Execution Screenshot

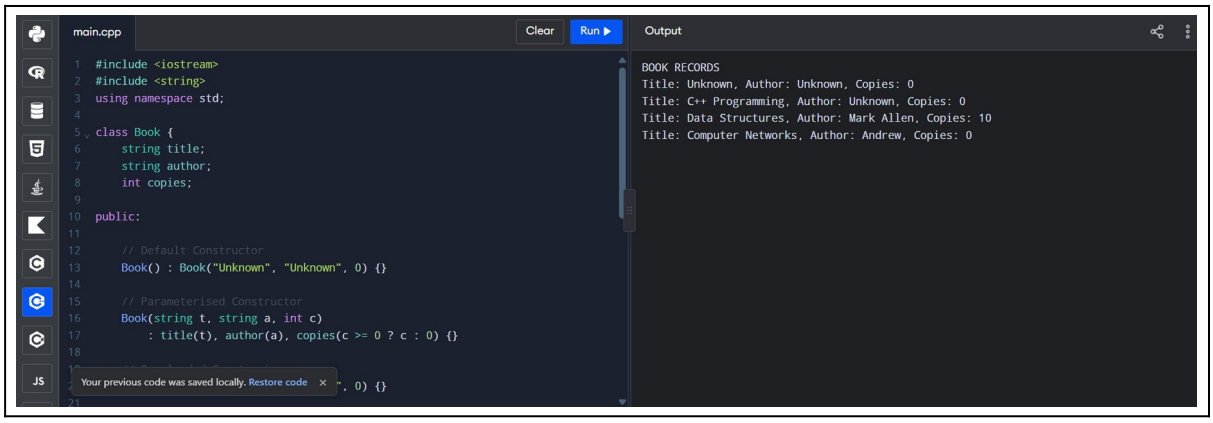


Figure 1.1: Output of the Book class using different constructors

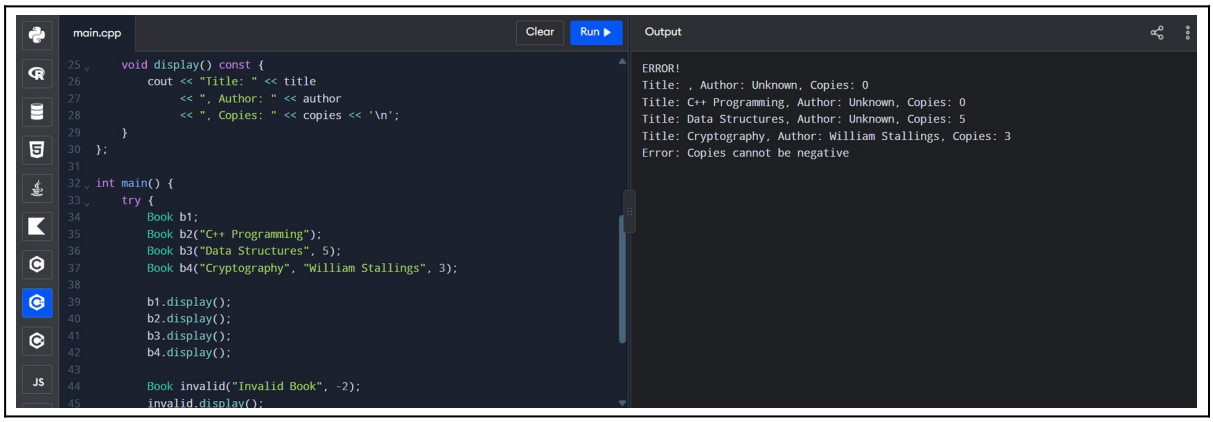


Figure 1.2: Output of the invalid copies test showing the error message

Figure 1.3

Compiler warning / error evidence (-Wreorder)

[Insert screenshot here: Q1_Wreorder_Warning.png]

Figure 1.3: Compiler warning generated for member initialisation order

1.7 Output Explanation

Figure 1.1 shows four Book objects. The first uses the default constructor, so the title and author are "Unknown" and copies is 0. The second gives only a title (C++ Programming), so the author

stays "Unknown" and copies is 0. The third gives a title, author and copies (Data Structures, Mark Allen, 10 copies) and the fourth gives Computer Networks by Andrew with 0 copies.

In Figure 1.2 the valid books are printed normally (Data Structures with 5 copies and Cryptography by William Stallings with 3 copies). When a book is created with -2 copies, the constructor reports the error "Copies cannot be negative" and the exception is caught in main().

Add anything else you want to explain, for example the compiler warning shown in Figure 1.3.

1.8 Design and Analysis

Explain why you chose these constructors, how the delegating constructor reduces repeated code, and why a member initializer list is used instead of assignment inside the constructor body.

Figure 1.4
Object state diagram for the Book class
[Insert diagram here]

Figure 1.4: Object state diagrams for the different constructors

1.9 Experimental Investigation

Invalid copies test

Describe the test inputs you used and how the program handled each of them.

Explicit constructor demonstration

Show how the explicit keyword stops implicit conversion and how the compiler reacts.

Compiler warning / error evidence

Explain the warning in Figure 1.3 and how it was fixed.

1.10 Alternative Concept and Complexity Analysis

Describe an alternative design (for example default arguments instead of several overloaded constructors) and compare the two in terms of time and space complexity.

Operation	Current approach	Alternative approach

1.11 Conclusion

Summarise what was achieved and what you learned from this question.

Q2. Copy Constructors: Shallow and Deep Copies (7 Marks)

2.1 Aim

To understand the difference between shallow copy and deep copy by writing a Prescription class that owns dynamically allocated data, with a user-defined copy constructor, copy assignment operator and destructor.

2.2 Problem Description

A Prescription class stores a list of medicine names in dynamically allocated memory. When one prescription is copied into another, changing a medicine in one prescription must not change the other. The program must also handle self-assignment safely.

2.3 Concepts Used

- Shallow copy and its problems (double delete, dangling pointer)
- Deep copy
- Copy constructor
- Copy assignment operator
- Destructor
- Self-assignment check
- Dynamic memory allocation (new / delete[])
- AddressSanitizer

2.4 Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode for your program here.

2.5 C++ Source Code

The source code should include the following parts:

- Shallow copy implementation (for comparison)
- Deep copy implementation
- Copy constructor
- Copy assignment operator
- Destructor
- main() with the self-assignment test and timing code

```
// Paste your C++ source code here
```

2.6 Program Execution Screenshot

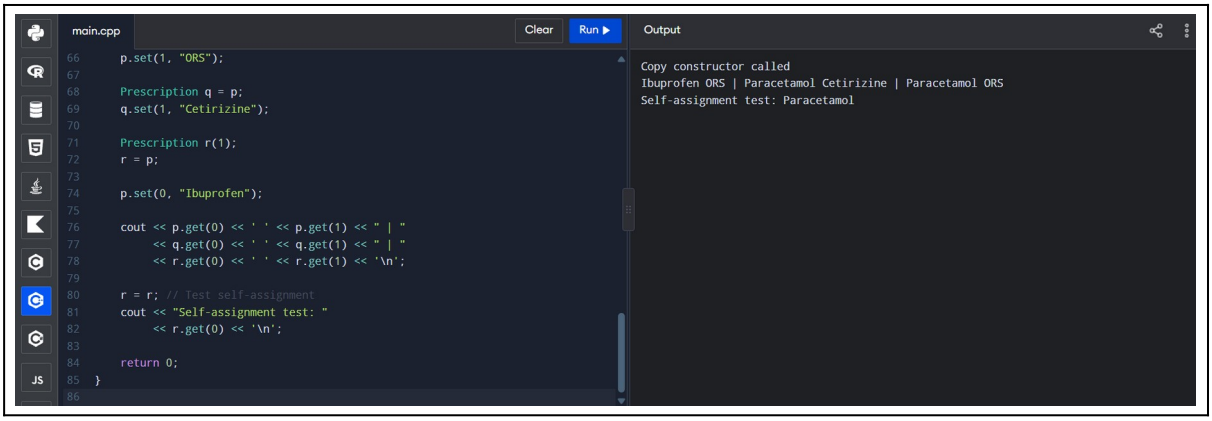


Figure 2.1: Output of the deep copy program with the self-assignment test

Figure 2.2

AddressSanitizer output

[Insert screenshot here: Q2_AddressSanitizer.png]

Figure 2.2: AddressSanitizer result for the deep copy program

2.7 Output Explanation

The message "Copy constructor called" appears once, when q is created from p. The line "Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS" shows p, q and r. p was changed to Ibuprofen after the copies were made, and q had its second medicine changed to Cetirizine, but r still holds Paracetamol and ORS. This shows that each object has its own memory, which is a deep copy.

The last line shows that r = r (self-assignment) leaves the object unchanged, so Paracetamol is still printed.

Explain the AddressSanitizer result in Figure 2.2 after you insert it.

2.8 Design and Analysis

Explain how the memory is laid out for the shallow copy and for the deep copy, and why the shallow copy causes a double free when both destructors run.

Figure 2.3
Memory diagram: shallow copy vs deep copy
[Insert diagram here]

Figure 2.3: Memory diagrams for shallow copy and deep copy

2.9 Experimental Investigation

Self-assignment test

The output in Figure 2.1 shows that self-assignment (r = r) leaves the data intact.

Explain how your operator= detects self-assignment.

Copy timing measurements

Record the values you measured on your own system.

Trial	Number of elements	Shallow copy (µs)	Deep copy (µs)

2.10 Alternative Concept and Complexity Analysis

Describe an alternative such as std::vector or smart pointers (std::unique_ptr / std::shared_ptr) that avoids writing the copy functions by hand, and compare complexity.

Operation	Current approach	Alternative approach

2.11 Conclusion

Summarise what was achieved and what you learned from this question.

Q3. Dynamic Constructors and Destructors (7 Marks)

3.1 Aim

To use constructors and destructors to allocate and release two-dimensional dynamic memory, using an Auditorium class for seat booking and cancellation, and to observe the order of constructor and destructor calls for stack and heap objects.

3.2 Problem Description

An Auditorium class holds a seating plan of a given number of rows and columns in a dynamically allocated 2D array. The constructor allocates the memory, the destructor releases it, and member functions book and cancel individual seats. Objects are created on the stack, inside a block and on the heap, and the destruction order is observed.

3.3 Concepts Used

- Dynamic constructor
- Two-dimensional dynamic array (int** / bool**)
- Destructor and memory release (delete[])
- Stack objects, block-scope objects and heap objects
- Order of constructor and destructor calls
- Static data member (event id counter)
- Memory leak and AddressSanitizer / leak checking

3.4 Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode for your program here.

3.5 C++ Source Code

The source code should include the following parts:

- Two-dimensional dynamic memory allocation in the constructor
- Destructor implementation
- Seat booking and cancellation functions
- Stack, block-scope and heap object demonstration in main()
- Static member for event numbering

```
// Paste your C++ source code here
```

3.6 Program Execution Screenshot

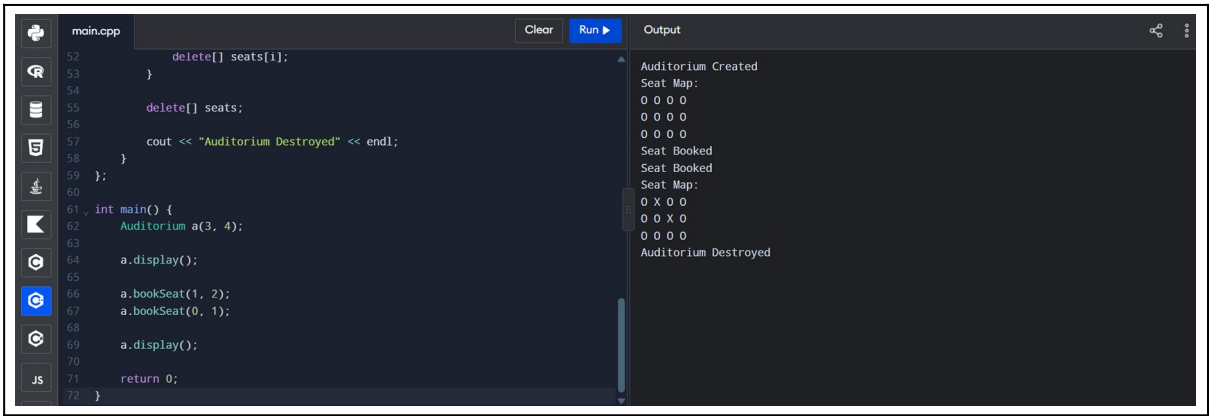


Figure 3.1: Output of the dynamic constructor program (seat map before and after booking)

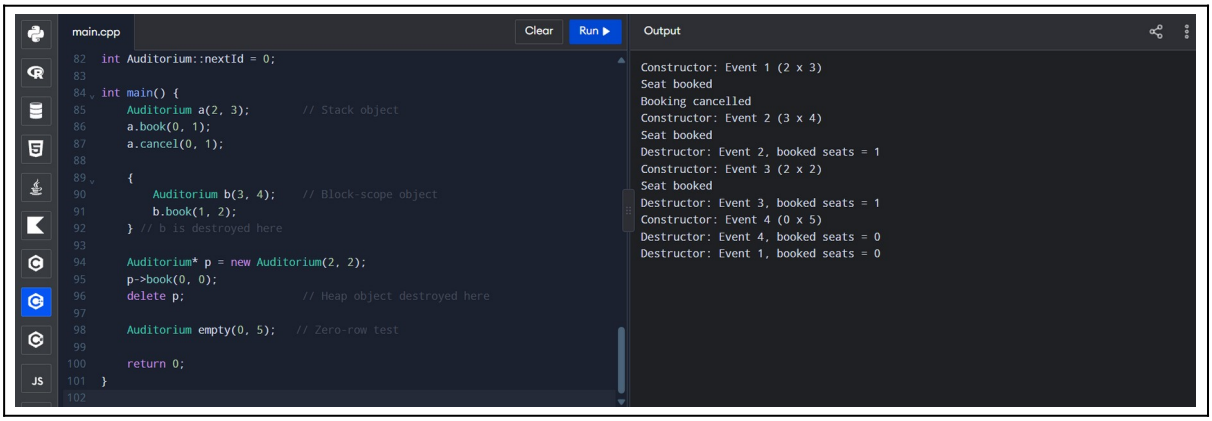


Figure 3.2: Constructor and destructor execution order for stack and heap objects

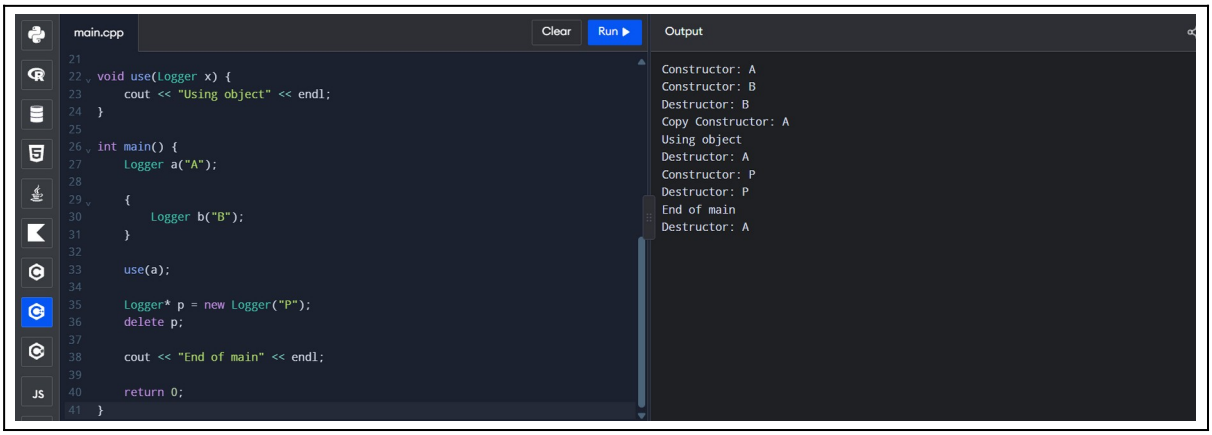


Figure 3.3: Destructor order for local, copied and heap objects

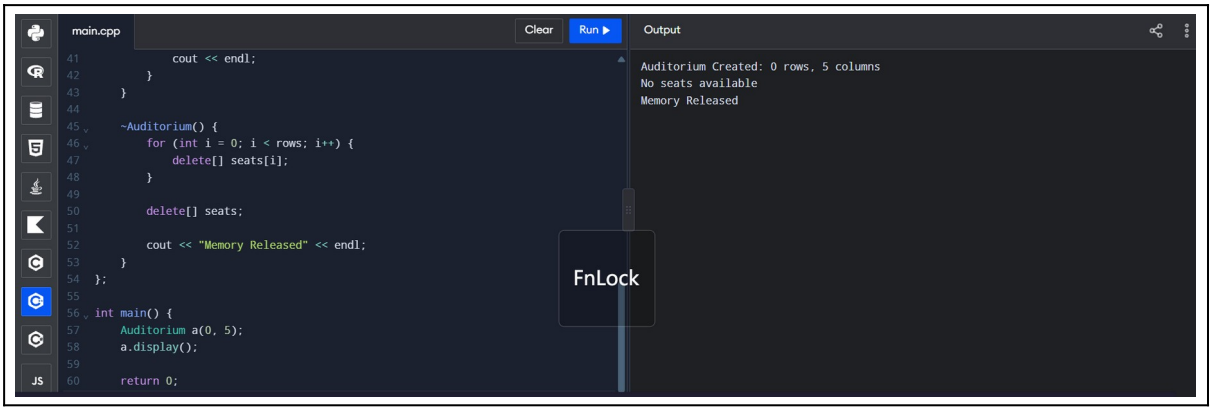


Figure 3.4: Auditorium with 0 rows and 5 columns

3.7 Output Explanation

In Figure 3.1 a 3 x 4 auditorium is created and the seat map is all zeros. After two seats are booked, X appears at row 0, column 1 and at row 1, column 2, and the destructor message appears at the end.

Figure 3.2 shows that the stack object (Event 1) is created first, a seat is booked and cancelled, then the block object (Event 2) is destroyed as soon as its block ends. The heap object (Event 3) is destroyed when delete is called. Event 4 is created with 0 rows. At the end of main() the destructors run in reverse order of creation, so Event 4 is destroyed before Event 1.

In Figure 3.3 objects A and B are created, B is destroyed at the end of its block, a copy of A is made when it is passed to use() and that copy is destroyed when the function returns. The heap object P is destroyed by delete, and A is destroyed last, after "End of main". Figure 3.4 shows that an auditorium with zero rows is handled without any error and the memory is released.

3.8 Design and Analysis

Explain how the 2D array is allocated (array of row pointers, then each row) and why the destructor must release the rows first and the row-pointer array afterwards.

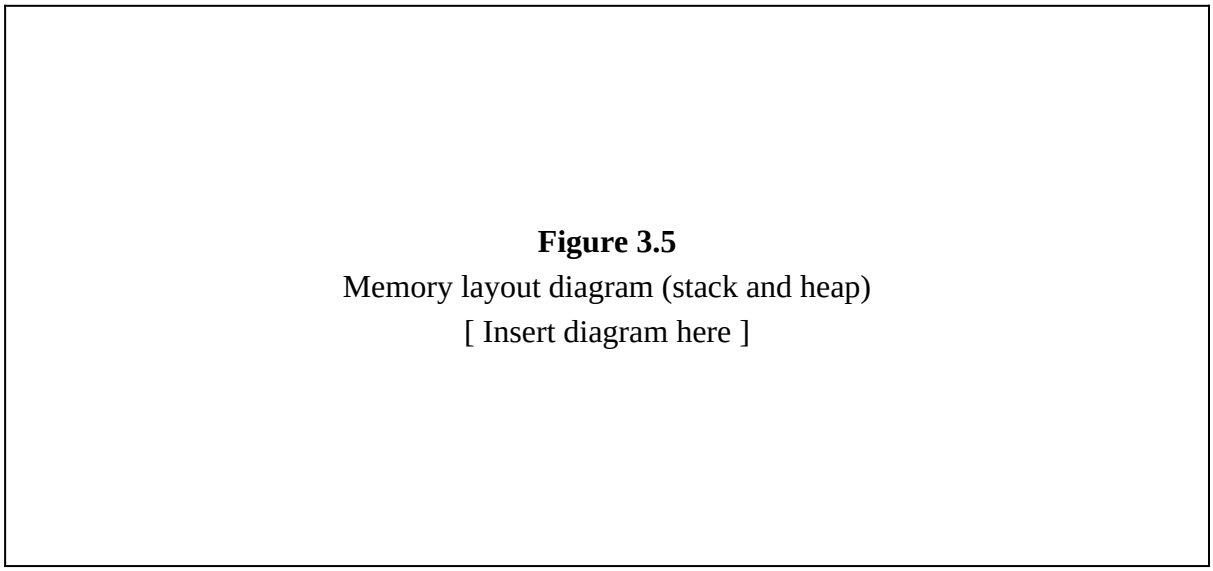


Figure 3.5: Memory layout of the Auditorium object

3.9 Experimental Investigation

Address Sanitizer results

Insert your AddressSanitizer / leak checker output here if you ran it, and describe the result.

Memory leak test

Figure 3.6 shows a program that allocates an array with new and does not release it. Figure 3.7 shows the corrected program that uses delete[] and sets the pointer to nullptr.

Explain the difference between the two programs.

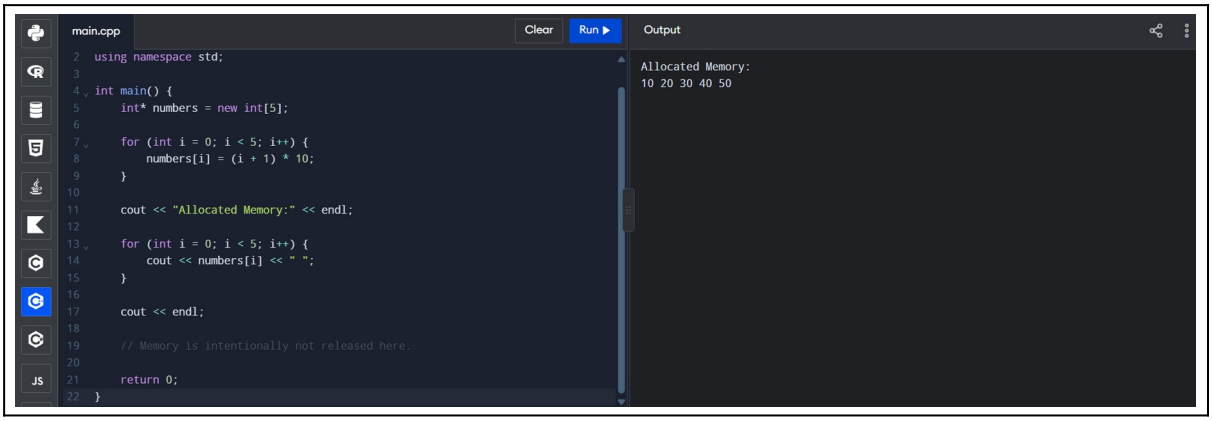


Figure 3.6: Program in which allocated memory is intentionally not released

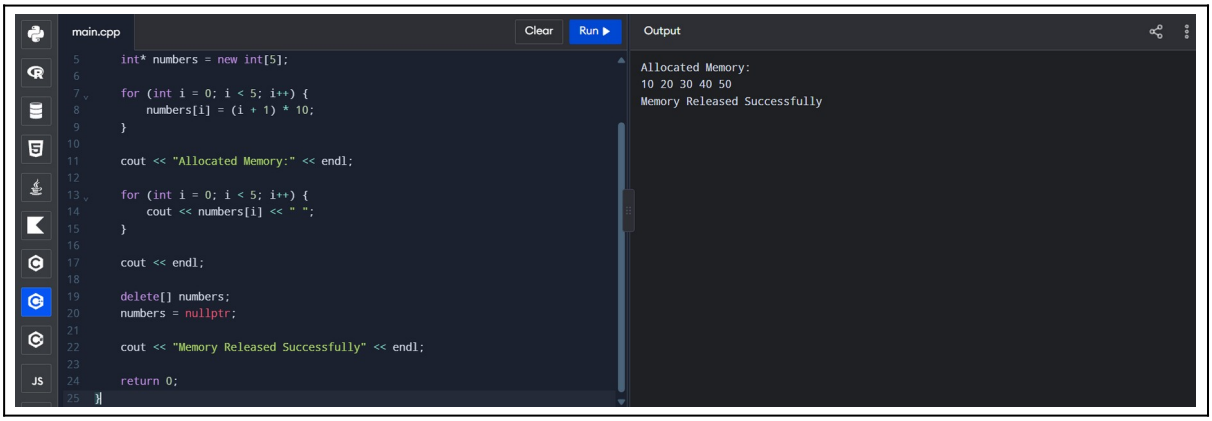


Figure 3.7: Corrected program that releases the memory

Memory measurements

Record the values you measured on your own system.

Test case	Rows x Columns	Memory used (bytes)	Remarks

3.10 Alternative Concept and Complexity Analysis

Describe an alternative such as std::vector<std::vector<int>> or a single contiguous array, and compare memory use and complexity.

Operation	Current approach	Alternative approach

3.11 Conclusion

Summarise what was achieved and what you learned from this question.

Q4. Arithmetic and Stream Operator Overloading (7 Marks)

4.1 Aim

To overload the arithmetic, comparison and stream operators for a Money class that stores rupees and paise.

4.2 Problem Description

A Money class stores an amount in rupees and paise. It must support addition, subtraction, +=, equality and less-than comparison, and input and output using the stream operators. When the paise exceed 99 they must carry over into rupees.

4.3 Concepts Used

- Operator overloading (+, -, +=, ==, <)
- Member and friend operator functions
- Stream insertion operator (<<) and extraction operator (>>)
- Returning objects by value and by reference
- Normalising paise into rupees (carry)
- const correctness

4.4 Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode for your program here.

4.5 C++ Source Code

The source code should include the following parts:

- Money class implementation
- Addition and subtraction operators
- += operator
- Equality and less-than operators
- Stream insertion and extraction operators
- main() with the paise carry test and negative subtraction test

```
// Paste your C++ source code here
```

4.6 Program Execution Screenshot

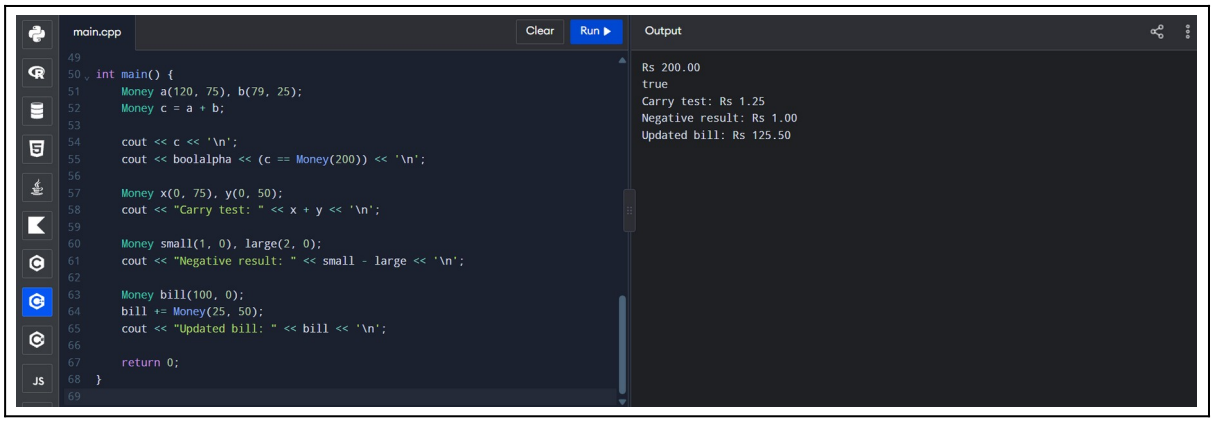


Figure 4.1: Output of the Money class operators

4.7 Output Explanation

In Figure 4.1, Rs 120.75 + Rs 79.25 gives Rs 200.00 and the comparison with Money(200) prints true. In the carry test, 75 paise + 50 paise gives Rs 1.25, so 125 paise is correctly converted to 1 rupee and 25 paise. The += operator adds Rs 25.50 to a bill of Rs 100.00 and the updated bill is Rs 125.50.

The negative subtraction test (Rs 1.00 - Rs 2.00) prints Rs 1.00.

Explain how your program represents or handles a negative result in this test.

4.8 Design and Analysis

Explain why some operators are written as friend functions, how the carry is calculated, and how the negative amount is stored.

4.9 Experimental Investigation

Paise carry test

Input: 75 paise + 50 paise. Output shown in Figure 4.1: Rs 1.25.

Negative subtraction test

Describe the input, the expected result and the result your program gave.

Test summary

Test	Input	Expected result	Actual output
Addition	120.75 + 79.25	Rs 200.00	Rs 200.00
Equality	200.00 == Money(200)	true	true
Paise carry	0.75 + 0.50	Rs 1.25	Rs 1.25
Negative result	1.00 - 2.00		Rs 1.00
+= operator	100.00 += 25.50	Rs 125.50	Rs 125.50

4.10 Alternative Concept and Complexity Analysis

Describe an alternative such as storing the whole amount as a single integer number of paise, and compare the two designs.

Operation	Current approach	Alternative approach

4.11 Conclusion

Summarise what was achieved and what you learned from this question.

Q5. Increment, Comparison and Subscript Operators (6 Marks)

5.1 Aim

To overload the prefix and postfix increment operators, the comparison operators and the subscript operator, using a Time class and a Playlist class.

5.2 Problem Description

A Time class stores time and supports prefix and postfix ++ and comparison operators, with rollover when a limit is crossed. A Playlist class stores song names and supports the [] operator with range checking, so an invalid index reports an error instead of accessing invalid memory.

5.3 Concepts Used

- Prefix increment (operator++())
- Postfix increment (operator++(int))
- Comparison operators (==, !=, <, >)
- Subscript operator [] (const and non-const versions)
- Range checking with out_of_range exception
- Returning references
- Initializer list constructor

5.4 Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode for your program here.

5.5 C++ Source Code

The source code should include the following parts:

- Time class
- Prefix and postfix increment operators
- Comparison operators
- Playlist class
- Subscript operator with range checking
- main() with the rollover test

```
// Paste your C++ source code here
```

5.6 Program Execution Screenshot

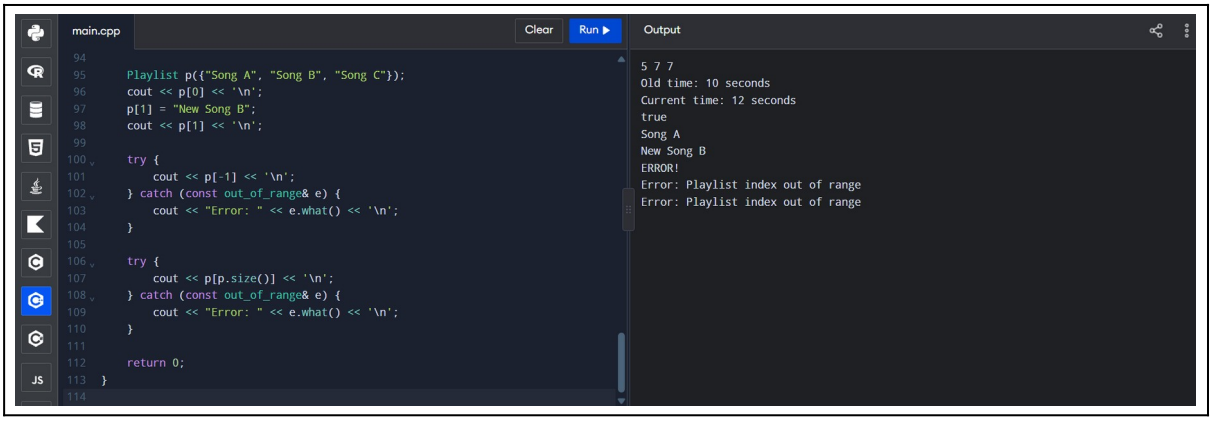


Figure 5.1: Output of the Time and Playlist programs

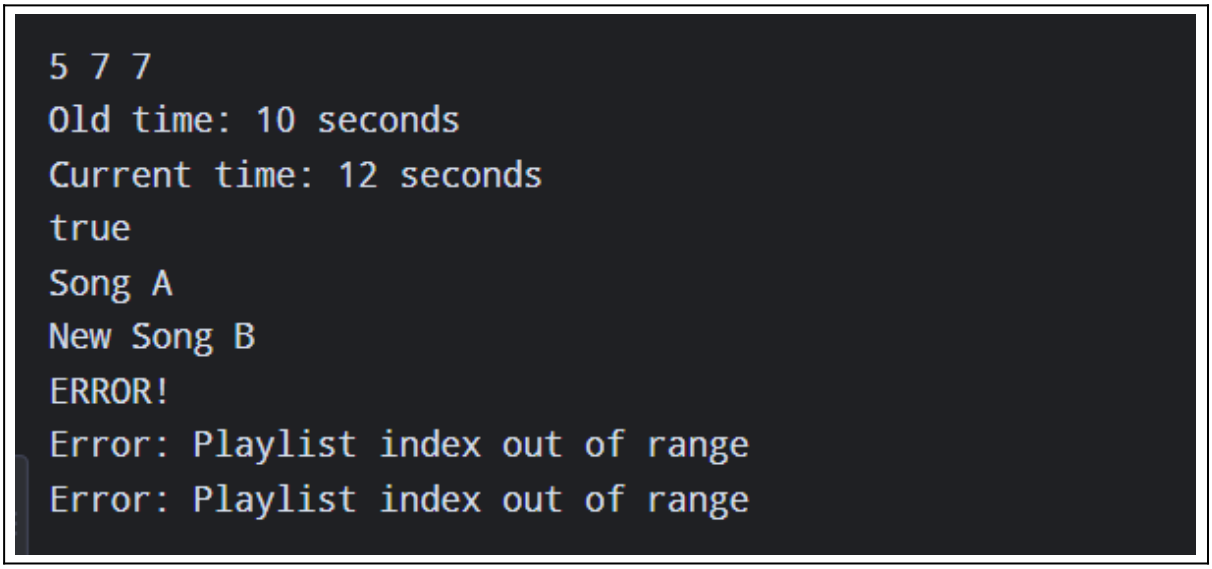


Figure 5.2: Output of the playlist bounds test (output panel)

5.7 Output Explanation

Figure 5.1 shows the output of the increment test (the line 5 7 7), the old time (10 seconds) and current time (12 seconds), and the result of the comparison (true).

For the Playlist, p[0] prints Song A and after the assignment p[1] = "New Song B" the element prints New Song B. Both p[-1] and p[p.size()] are outside the valid range, so the exception is caught and the message "Playlist index out of range" is printed twice (Figure 5.2).

Explain what each of the three values in the line "5 7 7" represents.

5.8 Design and Analysis

Explain why the postfix version takes a dummy int parameter, why it returns a copy while the prefix version returns a reference, and how the index is validated.

5.9 Experimental Investigation

Time rollover test

Describe the values used for the rollover test and the output obtained.

Performance measurements

Record the values you measured on your own system.

Operation	Number of iterations	Time taken (µs)	Remarks

5.10 Alternative Concept and Complexity Analysis

Describe an alternative such as std::vector with at() for bounds checking, and compare the two.

Operation	Current approach	Alternative approach

5.11 Conclusion

Summarise what was achieved and what you learned from this question.

Q6. Rules of Operator Overloading (7 Marks)

6.1 Aim

To apply the rules of operator overloading by writing a Text class that manages its own character memory, and to compare it with std::string.

6.2 Problem Description

A Text class stores a character string in dynamically allocated memory. It needs a copy constructor, copy assignment operator and destructor to manage this memory correctly, and overloaded operators for concatenation and comparison. The class is then compared with std::string in terms of performance.

6.3 Concepts Used

- Rule of three (copy constructor, copy assignment, destructor)
- Operator overloading rules (which operators can and cannot be overloaded)
- Concatenation operator (+)
- Comparison operators (==, !=)
- Self-assignment check
- Stream insertion operator
- Dynamic memory for character arrays
- AddressSanitizer

6.4 Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode for your program here.

6.5 C++ Source Code

The source code should include the following parts:

- Text class implementation
- Copy constructor
- Copy assignment operator
- Destructor
- Concatenation and comparison operators
- Self-assignment test in main()

```
// Paste your C++ source code here
```

6.6 Program Execution Screenshot

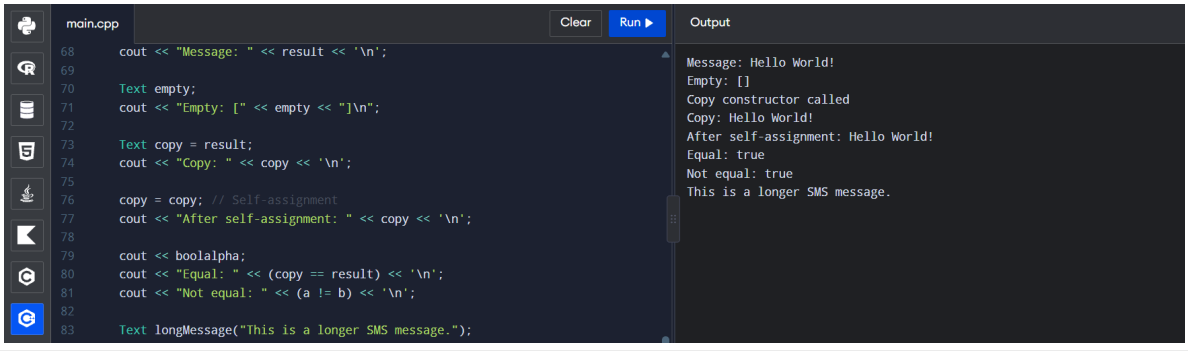


Figure 6.1: Output of the Text class program

Figure 6.2

AddressSanitizer output

[Insert screenshot here: Q6_AddressSanitizer.png]

Figure 6.2: AddressSanitizer result for the Text class

6.7 Output Explanation

In Figure 6.1 the message "Hello World!" is printed. An empty Text object prints as [] . The copy is created with the copy constructor (shown by the line "Copy constructor called") and prints Hello World!. After the self-assignment copy = copy the text is still Hello World!, so the assignment operator handles self-assignment safely. The comparison tests print Equal: true and Not equal: true, and the last line shows a longer message stored in a Text object.

Explain the AddressSanitizer result in Figure 6.2 after you insert it.

6.8 Design and Analysis

Explain the rules of operator overloading you followed (for example, operators that cannot be overloaded, keeping the usual meaning, member vs non-member functions) and why the rule of three is needed for this class.

6.9 Experimental Investigation

Operator overloading rules

List the rules of operator overloading and say how your Text class follows each of them.

Self-assignment test

The statement copy = copy was tested, and Figure 6.1 shows that the text is unchanged.

Explain how the check is done in your operator=.

Performance comparison with std::string

Record the values you measured on your own system.

Operation	Text class (µs)	std::string (µs)	Remarks

6.10 Alternative Concept and Complexity Analysis

Describe using std::string directly instead of a hand-written Text class and compare ease of use, safety and complexity.

Operation	Current approach	Alternative approach

6.11 Conclusion

Summarise what was achieved and what you learned from this question.

Q7. Complex Number Class (10 Marks)

7.1 Aim

To design a Complex number class with overloaded arithmetic operators and use it to calculate the series and parallel impedance of a resistor, inductor and capacitor (RLC) circuit.

7.2 Problem Description

Impedance in an AC circuit is a complex number: a resistor has Z = R + j0, an inductor has Z = 0 + jωL and a capacitor has Z = 0 - j/(ωC). A Complex class is needed to add, subtract, multiply and divide impedances, print them in rectangular form, find the magnitude, and handle division by zero.

Write your own interpretation of the problem (circuit values, frequency, what has to be calculated).

7.3 Concepts Used

- Class design with real and imaginary parts
- Constructors with default arguments
- Overloading +, -, * and /
- Rectangular representation (a + jb)
- Magnitude = sqrt($a^2 + b^2$)
- Series impedance (Z1 + Z2 + Z3)
- Parallel impedance (Z1 x Z2 / (Z1 + Z2))
- Exception handling (domain_error) for division by zero
- std::complex<double>

7.4 Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode for your program here.

7.5 C++ Source Code

The source code should include the following parts:

- Class design (data members and member functions)
- Constructors
- Arithmetic operator overloading (+, -, *, /)
- Rectangular representation (stream insertion operator)
- Magnitude calculation
- Series and parallel impedance functions
- Division-by-zero check and main()

```
// Paste your C++ source code here
```

7.6 Program Execution Screenshot

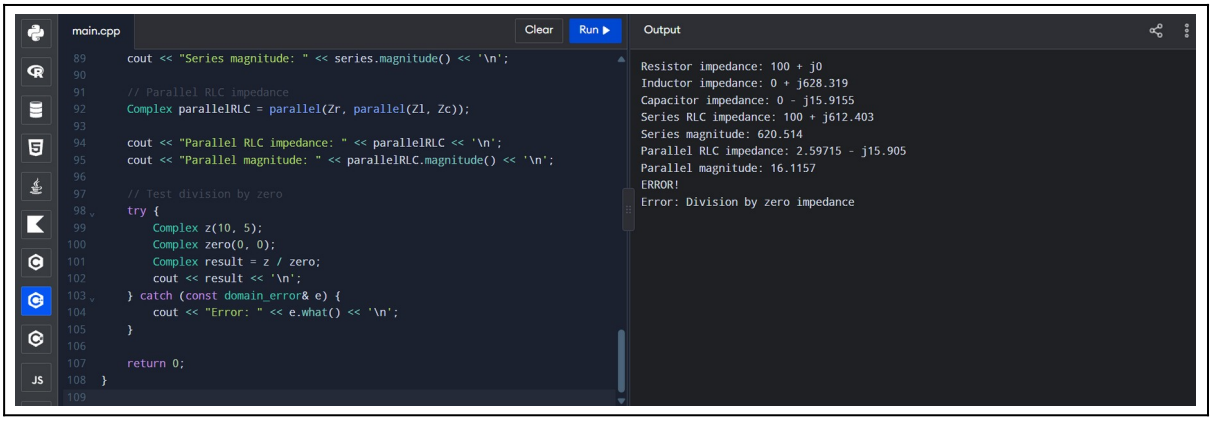


Figure 7.1: Output of the Complex class impedance program

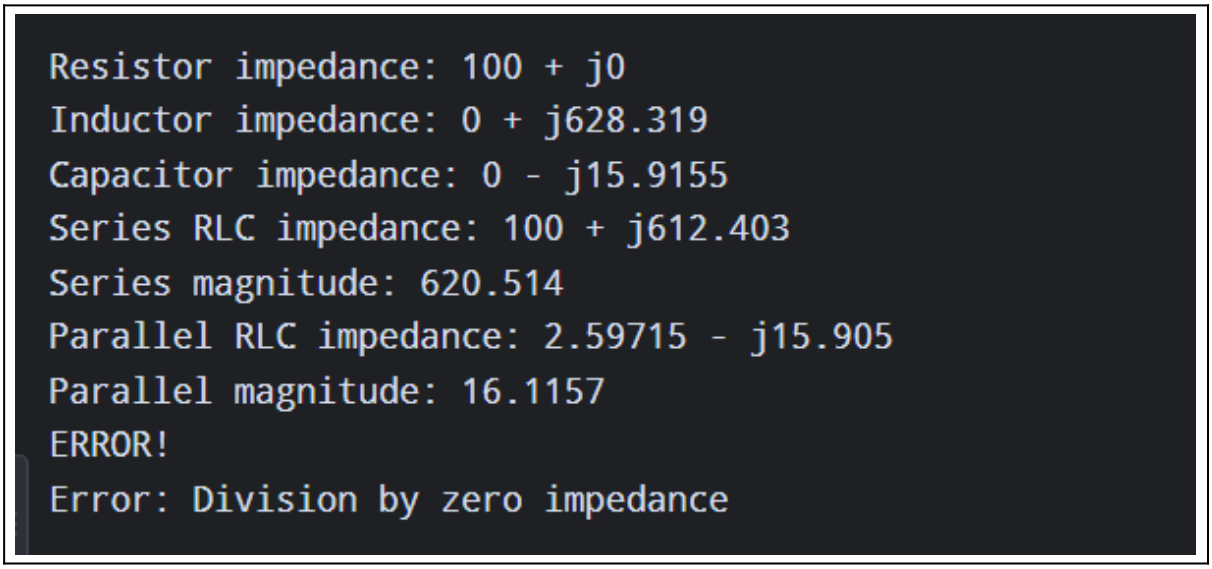


Figure 7.2: Division by zero impedance error (output panel)

7.7 Output Explanation

In Figure 7.1 the resistor impedance is 100 + j0, the inductor impedance is 0 + j628.319 and the capacitor impedance is 0 - j15.9155. The series RLC impedance is 100 + j612.403 with a magnitude of 620.514. The parallel RLC impedance is 2.59715 - j15.905 with a magnitude of 16.1157.

The division by zero test divides 10 + j5 by 0 + j0. The class detects the zero denominator and throws a domain_error, which is caught in main() and printed as "Error: Division by zero impedance" (Figure 7.2).

7.8 Design and Analysis

Explain how the class is designed, why the operators return new Complex objects, and how the parallel impedance of three components is obtained by combining two components at a time.

Figure 7.3
Class diagram or RLC circuit diagram
[Insert diagram here]

Figure 7.3: Complex class design / RLC circuit

7.9 Experimental Investigation

Series impedance calculation

Series: Z = Zr + Zl + Zc = 100 + j(628.319 - 15.9155) = 100 + j612.403 (matches the program output).

Parallel impedance calculation

Zl in parallel with Zc = (j628.319)(-j15.9155) / (j612.403) = -j16.329 (approximately). Zr in parallel with this result gives about 2.597 - j15.905, which matches the program output.

Manual verification

Magnitude of the series impedance = sqrt(100^2 + 612.403^2) = sqrt(385037.5) = 620.51 (approximately), which matches the program output of 620.514.

Add your own hand calculation here and compare it with the program output.

Division-by-zero test

Describe the test and explain the behaviour shown in Figure 7.2.

Comparison with std::complex<double>

Run the same calculations with std::complex<double> and compare the results.

Quantity	Own Complex class	std::complex<double>
Series impedance		
Series magnitude		
Parallel impedance		
Parallel magnitude		

7.10 Alternative Concept and Complexity Analysis

Describe using std::complex<double> from the <complex> header instead of a user-defined class and compare the two in terms of effort and complexity.

Operation	Current approach	Alternative approach

7.11 Conclusion

Summarise what was achieved and what you learned from this question.

Overall Learning Outcomes

Write the main things you learned from this assignment (constructors, destructors, copy control, operator overloading, memory management).

Final Conclusion

Write the overall conclusion of the assignment.

GitHub Repository Link

Repository link: https://github.com/192524062simats-ops/DSA0103c-/tree/main/c%2B%2B%20assignment

References

[1] Bjarne Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley, 2013.

[2] cppreference.com, C++ reference documentation, https://en.cppreference.com

[3] Add your lecture notes, textbook or website references here.

[4]